

Basics of Python

Programming refers to the process of designing and constructing a program (a set of instructions for the computer) for a specific purpose. To put it simply, programming is about describing the steps to perform a certain calculation.

For example, when you want to find the sum of 2 and 3, you can write and execute the following program to obtain the answer. This is a simple example, but still a valid program.

```
2 + 3
```

As a more complex example, here is a program to calculate the sum of numbers from 1 to 10. (We will learn more details in the following sections).

```
s = 0
for n in range(1,11):
    s = s + n
s
```

When programming, you use some **programming language**. Programming languages have rules like "writing program A executes process B". These rules are called **syntax** or **grammar**, which varies depending on the programming language used. Learning programming starts with understanding this grammar.

In this course, we will use the language Python for programming because there are plenty of tools available for data analysis and machine learning. In fact, many people use Python in that field. Python is also a relatively easy language for programming beginners to understand. Therefore, in this chapter, we will learn the basic grammar of Python.

1.1. Approach to Programming

We will now start programming with Python, so let me briefly explain the process. You are probably using **Google Colaboratory** (or a similar tool) to read this file. Google Colaboratory provides a Python execution environment where you can write Python programs in boxes called **cells** and run them by pressing Shift + Enter.

There are several types of cells, but the one used to execute programs is called a **Code cell**. When you write a program and execute it, the execution result is displayed.

You can also leave notes in the notebook. The cell used for this purpose is called a **Text cell**. By using Markdown syntax, you can create well-formatted text.

Now, let's run the program we just wrote. If the correct output is returned, that's OK.

```
2 + 3
```

```
s = 0
for n in range(1,11):
    s = s + n
s
```

1.2. Comments

In addition to Jupyter Notebook, there are various Python execution environments available. For example, the Python interactive shell returns the execution results for the input program. However, it does not typically save the input program or output results, so the content disappears when the shell is closed. In contrast, Jupyter Notebook allows you to save both the program and its execution results, so there is no issue even when you close the notebook.

However, even if the program is retained, its meaning may become unclear. When writing a program, it is useful to leave moderate notes to refer back to later. In many programming languages, you can add **comments** to the program, and the parts written as comments are ignored during program execution.

In Python, the part to the right of `#` is considered a comment. Let's confirm this with an example.

```
# Addition
2 + 3 # Addition
# Comment...
```

✓ 1.3. Dealing with Errors

In programming, **errors** are inevitable. Errors occur when writing programs that do not follow the syntax or when performing some kind of invalid operation. When an error occurs, the program execution stops at that point, and an error message is displayed indicating where and why the error occurred. You will likely encounter many errors in the future, but the first step is to try to decipher the error message. If you don't understand its meaning, try searching on the internet. There are surely people who have struggled with similar errors.

✓ 1. Python Grammar I

Related explanation video: https://www.youtube.com/playlist?list=PLiEF5y87wttyc3mVmmRpUcAi_a74w52CI

✓ 1.4. Operators

✓ 1.4.1. Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations in Python.

In programming, **operators** instruct the computer to evaluate new values. Specifically, arithmetic operators such as addition and subtraction perform calculations between numbers, similar to mathematics. In Python, addition and subtraction use symbols common in math, while multiplication, division, and exponentiation have their own unique symbols.

```
# Addition
print(2 + 3)
```

5

```
# Subtraction
print(10 - 2)
```

8

```
# Multiplication
print(3 * 2)
```

6

```
# Division
print(3 / 2)
```

1.5

```
# Integer division
print(5//2)
```

2

```
# Modulo operation (remainder)
print(5 % 2)
```

```
# Exponential operation
print(10 ** 3)
```

✓ 1.4.2. Operator Precedence in Arithmetic

When there are multiple operators, they are evaluated in order of higher precedence. The precedence of arithmetic operators is as follows: exponentiation > multiplication and division > addition and subtraction. Just as in mathematics, where multiplication is done before addition, you can use parentheses to change the order of evaluation.

```
# Priority: Multiplication > Addition
print(1 + 2 * 3)
```

```
# Priority: addition inside parentheses > multiplication
print((1 + 2) * 3)
```

```
# Priority: Exponentiation > Multiplication
print(2 * 2 ** 3)
```

16

1.5. Variables

Next, let's talk about **variables**. Variables are like boxes where you can store the results of calculations.

Assigning a value to a variable is called **assignment**, and using that value is called **referencing**. You can assign values of various data types, not just numbers. (§1.6 Data Types)

In the example below, a variable with the **variable name** `x` is used.

In programming languages, the `=` sign is not used to denote "equality". Instead, it is used to "assign" the value on the right side to the left side.

```
# Assign 1 to x and print x
x = 1
print(x)
```

Once a value is saved, it can be used multiple times and will remain until it is overwritten. Overwriting the value of a variable is called **reassignment**. When reassigning, you can write it in the same way as assigning. It is also possible to reassign using the variable itself.

```
# Reassigning x+2 to x
x = x + 2
print(x)
```

Let's explain the second line of the previous example. Here, we first calculate `x+2` on the right side, and then we perform the operation of assigning the result to `x` on the left side. At the point of calculating the right side, the value of `x` is 1, so the result of the calculation on the right side is 3, and this value 3 is assigned to `x` on the left side. In other words, you can interpret this as adding 2 to itself. This is a common practice to update variable values while progressing with calculations.

1.6. Data Types

The types of values handled in a program are called **data types**. Major data types include int (integer), float (decimal), str (string), bool (boolean), and so on.

Data Type	Description	Example
int	Integer	1
float	Decimal (Floating Point)	3.14
str	Text Data (String)	'Python'
bool	Boolean	True
list	List	[0, 1, 2]

You can examine the data type of a value assigned to a variable using the `type()` function. By inputting the variable in parentheses, the `type()` function will output the data type of the variable.

1.6.1. Numeric Values : int Type (Integer) and float Type (Decimal)

There are two types for handling numeric values: **int** type and **float** type. When a decimal point is included in a number, it becomes a float type.

When performing operations between int and float types or when dividing two int types, the output will be a float type.

```
# int type (integer)
num = 1
print(type(num))
```

```
<class 'int'>
```

```
# float type (decimal)
num = 1.0
print(type(num))
```

```
<class 'float'>
```

```
# Calculation between int and float -> float type
num = 1 + 1.0
print(type(num))
```

```
<class 'float'>
```

```
# Divide two int types
num = 1 / 1
print(type(num))
print(num)
```

```
<class 'float'>
1.0
```

✓ 1.6.2. Text Data : str Type (String)

The `str` type is a data type that handles strings. When assigning a value to a variable as `str`, you enclose it in either single quotes `' '` or double quotes `" "`. When the addition operator for numbers is applied to `str` types, it connects the strings. Also, the `len()` function can be used to determine the length of a string.

```
# Assigning a string
string = 'Python'
print(type(string))
print(string)
```

```
<class 'str'>
Python
```

```
str1 = 'Hello'
str2 = 'World'
print(str1 + "," + str2) # Adding a comma between
```

```
Hello,World
```

```
# Repeat
text = "Hello "
repeated = text * 3
print(repeated)
```

```
Hello Hello Hello
```

```
# Length of the string
string = 'Python'
print(len(string))
```

```
6
```

```
# Get the first character
text = "Apple"
part = text[0] # 0th index
print(part)
```

Element of the string	A	p	p	l	e
Index	0	1	2	3	4

```
# Extract a substring from a string
text = "Apple"
part = text[0:3] # Characters from index 0 to 2 (index 3 is not included)
print(part)
```

data_science_lecture_series.ipynb

Element of string	A	p	p	l	e
Index	0	1	2	3	4

```
# f-string
name = "Matsuo"
age = 25
message = f"My name is {name} and I am {age} years old." # You can embed variables inside {}.
print(message)
```

1.6.3. bool Type (Boolean)

The boolean data type has two values: True and False, representing true and false, respectively. Boolean data type is useful for conditional statements, similar to using "yes" and "no".

```
yes = True
print(type(yes))

no = False
print(type(no))
```

```
<class 'bool'>
<class 'bool'>
```

1.6.4. Type Conversion

Changing the data type of a value assigned to a variable is known as **type conversion**. The `int()` function is provided for converting to the int type. Additionally, there are other functions such as `float()`, `str()`, and `bool()`.

When an int type is input into the `float()` function, only the integer part is output. Also, it is important to note that entering characters like 'abc' into int or `float()` functions will result in an error.

```
# str type
x = '3.14' # Not a numerical value but a string
print(x)
print(type(x))
```

```
3.14
<class 'str'>
```

```
# Converting from str type to float type
y = float(x)
print(y)
print(type(y))
```

```
3.14
<class 'float'>
```

```
# Convert from float to int
z = int(y)
print(z)
print(type(z))
```

```
3
<class 'int'>
```

A useful feature is that boolean values can be converted to numbers. True becomes 1, and False becomes 0. Conversely, when converting numbers to boolean values, 0 becomes False, while any nonzero number becomes True.

```
# Converting from bool to int
# True -> int() -> 1
x = True
print(int(x))
```

```
# Converting from bool to int
# False -> int() -> 0
x = False
print(int(x))
```

```
# Converting from int to bool
# 2 -> bool() -> True
print(bool(2))
```

```
# Convert from int to bool
# 0.0 -> bool() -> False
print(bool(0.0))
```

```
# Variable num: Is it odd (cannot be divided by 2)? -> True for odd, False for even.
x = 5
is_odd = bool(x % 2) # Since the remainder is not 0, it converts to True.
print(is_odd)
```

✓ 1.7. Practice Questions

(1-1) Write a program that outputs a string of your choice.

```
# WRITE ME!
```

Example: When $x = \text{"Hello"}$ and $y = \text{"Mr. Matsuo"}$, print $x + y$.

```
# WRITE ME!
```

(1-2) Please write one or more programs performing any operations of your choice using arithmetic operators (+, -, *, /, %, **, //).

```
# WRITE ME!
```

Example: Calculate the total price including 10% consumption tax when $x = 2000$ yen.

```
# WRITE ME!
```

(1-3) Let's assume that an integer is stored in the variable x . Create a program that will display (print) 0 if x is even, and 1 if x is odd.

```
x = 1 # Input example
```

```
# WRITE ME!
```

(1-4) Create a program to calculate BMI from height and weight. $\text{BMI} = \text{weight}(\text{kg}) / (\text{height}(\text{m}))^2$

```
# Input example
weight = 50 # kg
height = 165 # cm

bmi = # write me
print(bmi)
```

✓ Additional Information: Naming Variables

You are free to choose variable names, but generally they are composed of alphabet letters, numbers and _ (underscore). While you can use single letters like a or b, using meaningful strings will make it easier for yourself and others to understand the contents of the variables. However, you need to be careful as there are some variable names that cannot be used. For example, you cannot use numbers at the beginning of a variable name. Also, words that are reserved in the program (e.g. class, if, for, etc.) cannot be used. The following are examples of variable names that are not allowed. If you try to execute with variable names like the ones below, an error will occur.

```
'''
※Variables that cannot be used as variable names※
'''
# Variable name starts with a number
2024_data = 1000
```

```
class_name = 'A'
```

You can verify reserved words by executing the following code:

```
import keyword
print(keyword.kwlist)
```

✓ 2. Python Grammar II

Related explanation video: https://www.youtube.com/playlist?list=PLiEF5y87wtYyc3mVmmRpUcAi_a74w52CI

✓ 2.1. Collections

In previous examples, we defined variable names and values on a one-to-one basis. On the other hand, Python provides collections that allow you to **store multiple values in a single variable**. Collections include lists and tuples that manage values in order, as well as dictionaries that manage values by keywords.

✓ 2.2. Lists

A **list** is a data type that can manage values in order. By using square brackets [], values can be assigned to variables by listing them with commas (.). Elements can include data types such as strings and numbers.

```
menu = ['Salad', 'Soup', 'Steak', 'Cake']
```

Each value in a list is called an **element**, and the order of the elements is referred to as the **index**. The index starts at 0 for the first element, and increases sequentially as 1, 2, 3, and so on. It is also possible to count from the last element, starting with -1, -2, -3, and counting back to the first element. By specifying the index in square brackets after the variable name of the list, it is possible to extract or replace elements.

```
# Extracting the second element of the list
print(menu[1])
```

```
Soup
```

```
# Extracting the last element of the list
print(menu[-1])
```

```
# Change the last element of the list
menu[-1] = 'pudding'
print(menu)
```

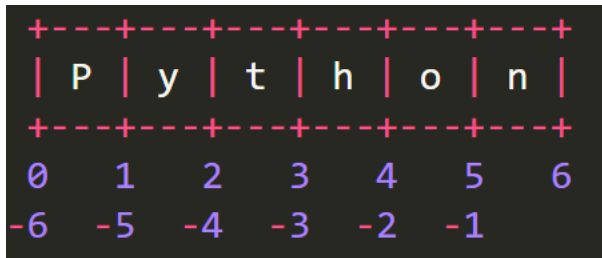
```
['Salad', 'Soup', 'Steak', 'pudding']
```

You can extract a portion of a list by specifying an index range which is called **slicing**. Similar to extracting elements, you can use square brackets after the list variable name, with the starting index on the left side of the colon (:) and the ending index on the right side. In this case, the starting point is included in the slice, but the ending point is not. For example, the slice notation [x:y] represents indices greater

than or equal to x, but less than y. If x or y is omitted, the selection continues to the remaining range.

A good way to learn how to use slicing is to consider indices pointing between elements, with the leftmost element starting at 0, as described in the Python tutorial.

You can imagine it as shown in the diagram below.



(Reference)(Python Tutorial): <https://docs.python.org/3/tutorial/introduction.html#strings>

```
# Assigning a list of numbers
numbers = [10, 20, 30, 40, 50]
```

Element in the list	10	20	30	40	50
Index	0	1	2	3	4

```
# Example of slicing
print(numbers[0:2])
```

```
# Specify a start and end point for a slice
print(numbers[1:3])
```

```
# When only the end point is specified in the slice
print(numbers[: -1])
```

```
# Slice without specifying indices (all elements)
print(numbers[:])
```

When you want to add or remove elements in a list, you can use the dot (.) following the variable name of the list, followed by **append (to add elements at the end)** or **remove (to delete elements)**. When using remove and there are duplicate elements in the list, the first occurrence of the element will be removed.

```
menu = ['salad', 'soup', 'steak', 'cake']
```

```
# Add an element
menu.append('pudding')
print(menu)
```

```
# Remove element
menu.remove('soup')
print(menu)
```

In Python, convenient functions for numerical calculations with lists are provided. In Python's built-in functions, you can perform calculations by inserting values inside the parentheses after the function name. The "len" in `len()` is an abbreviation for "length" and is commonly used with other data types as well.

```
# List of numerical values
score = [40, 60, 80]
```

```
# Output the number of elements in the list
print(len(score))
```

```
# Print the sum of the list of numbers
print(sum(score))
```

```
# Calculate the average of a list of numbers
print(sum(score)/len(score))
```

```
# Output the maximum value of a numerical list
print(max(score))
```

```
# Output the minimum value of a list of numbers
print(min(score))
```

Lists can manage two-dimensional tabular data using double square brackets. You can extract elements or lists by specifying row and column numbers. It is also possible to create lists of three or more dimensions.

```
# Assigning a 2D list
table = [['24/3', '24/4', '24/5'], [80, 100, 90]]
```

		(Column 0)	(Column 1)	(Column 2)
(Row 0)	Date	3/24	4/24	5/24
(Row 1)	Score	80	100	90

```
# Extracting element at row 0, column 2
print(table[0][2])
```

```
24/5
```

```
# Extracting element at row 1 column 1
print(table[1][1])
```

```
100
```

2.3. Tuples

A tuple, like a list, is a collection of multiple elements and is called a **tuple** in other programming languages.

In Python, data is stored in the form of `(element, element, ...)` for tuples.

```
# Double
x = (1, 2, 3)
print(x)
```

When it comes to grouping the same type of data together, lists are commonly used, while tuples are often used to group different types of data. Although tuples have a similar data structure to lists and can perform similar operations, there are subtle differences between the two. In the following sections, we will learn about tuples by comparing their similarities and differences with lists.

In tuples, like in lists, you can access elements using indices.

Tuple Element	1	2.0	'3'
Index	0	1	2

```
# Get the reference of elements
x = (1, 2.0, '3')
print(x[0])
```

```
1
```

```
# Slice (reference)
x = (1, 2.0, '3')
print(x[0:2])
```

```
(1, 2.0)
```

However, you cannot assign elements to a tuple. Unlike lists, once defined, tuples do not allow modifications such as adding or removing elements. While lists are mutable, meaning they can be changed, it is important to note that tuples are immutable, meaning they cannot be changed.

```
x = (1, 2.0, '3')
x[1] = 3.0
```

Actually, you can omit the parentheses when creating a tuple. Therefore, the following two examples can be considered the same code.

```
# Example 1 using parentheses
x = (1, 2.0, '3')
print(x)
```

```
# Example 2: Omitting parentheses
x = 1, 2.0, '3'
print(x)
```

Example 2 can also be thought of as assigning multiple values to a single variable. This is called **multiple assignment**. There are related terms such as **packing** and **unpacking**, but we will leave the introduction of these terms here for now. If you are interested in learning more advanced content, it would be good to look up these terms.

2.4. Dictionaries

A dictionary is a collection that associates a label called a **key** with each element of a list, known as an **associative array** in other programming languages.

In Python, data is stored in the format `{<key>: <value>, ...}`.

```
# Dictionary
menu = {'apple':100, 'banana':80, 'orange':120}
print(menu)
```

In lists, we access each element by index, while in dictionaries, we access values using keys. Similar to lists, we can reference and assign values.

```
# Accessing elements
print(menu['apple'])
```

If you assign a value to a key that is not in the dictionary, it will be added as a new element.

```
# Add a key and value
menu['grape'] = 150
print(menu)
```

And to get rid of an item, we use the `del` statement with the key we want to remove.

```
# Remove a key and value
del menu['apple']
print(menu)
```

There are also ways to retrieve keys, values, and entire elements from a dictionary.

We can use the dictionary's `keys()` method, `values()` method, and `items()` method.

```
# Get keys
x = {'a': 1, 'b': 2, 'c': 3}
print(x.keys())
```

```
# Get elements
x = {'a': 1, 'b': 2, 'c': 3}
print(x.values())
```

```
x = {'a': 1, 'b': 2, 'c': 3}
print(x.items())
```

2.5 Practice Questions

(2-1) Write a list that includes all prime numbers less than 10.

Start coding or [generate](#) with AI.

(2-2) Write a tuple that includes all prime numbers less than 10.

Start coding or [generate](#) with AI.

(2-3) Write a dictionary whose key is the string `'name'` and whose value is a string containing your own name.

Start coding or [generate](#) with AI.

3. Python Grammar III

Related explanation video: https://www.youtube.com/playlist?list=PLiEF5y87wttyc3mVmmRpUcAi_a74w52Cl

3.1. Conditional Branching (if Statement)

Conditional branching is a feature in a program where the flow of processing is altered based on certain conditions. If a particular condition is true, certain actions are taken, and if false, alternate actions are executed.

In Python, this is expressed using `if`, `elif`, and `else`. The format is shown below:

```
if <condition0>:
    <(actions to be performed when condition0 is True)>
elif <condition1>:
    <(actions to be performed when condition0 is False and condition1 is True)>
else:
    <(actions to be performed when all conditions are False)>
```

The `if` statement executes actions if the condition is `True`. You can add an `else` clause to specify code that runs when the condition is `False`. Furthermore, if you want to specify multiple conditions, you can use `elif` clauses in conjunction. You can have multiple `elif` clauses as needed.

It is important to note that **you must indent four spaces before the actions (to be performed when the condition is True/False)**. In Python, this is referred to as **indentation**. Incorrect indentation leads to an error.

Let's take a look at an example. In the example below, we calculate the absolute value of the variable `x` and store the result in `y`.

```
# Calculate the absolute value
x = -1
if x < 0:
    y = -x
else:
    y = x

print(y)

1
```

While not explicitly stated above, it is also possible to execute multiple consecutive processes. In this case, you need to insert an indent before each of those processes. In programming, a grouping of processes is generally referred to as a **block**, and in Python, any section with an inserted indent is considered a block and processed as such.

The following two examples may appear similar, but they are actually completely different codes. Pay attention to the indentation and see how it affects the code.

```
# Example 1 *Note the indentation level
x = 1
if x < 0:
    x = - x
    x = - x
print(x)
```

1

```
# Example 2
x = 1
if x < 0:
    x = - x
x = - x
print(x)
```

-1

Let's take a look at an example using the `elif` statement. In the following code snippet, since `x` is not less than 0, the first `if` statement is not executed, and the program moves to the `elif` statement. As `x` is less than 10, the `elif` statement is executed, resulting in `y` being assigned a value of -1.

```
# Example of elif
x = 1
if x < 0:
    y = 0
elif x < 10:
    y = -x
else:
    y = x
print(y)
```

Next, let's take a look at comparison operators used in conditional branching. Comparison operators include the following:

Comparison Operator	Description
<code>x == y</code>	<code>x</code> is equal to <code>y</code>
<code>x != y</code>	<code>x</code> is not equal to <code>y</code>
<code>x > y</code>	<code>x</code> is greater than <code>y</code>
<code>x < y</code>	<code>x</code> is less than <code>y</code>
<code>x >= y</code>	<code>x</code> is greater than or equal to <code>y</code>
<code>x <= y</code>	<code>x</code> is less than or equal to <code>y</code>

```
x = 10
y = 20

if x < y:
    print(f"{x} is less than {y}")
elif x == y:
    print(f"{x} is equal to {y}")
else:
    print(f"{x} is greater than {y}")
```

The `f` in `f-string` stands for **format**. It is convenient for embedding variables within the arguments of `print`.

Now, let's take a look at logical operators that are used for conditional branching, in addition to comparison operators. Here are some of the logical operators:

Logical Operator	Explanation
<code>x and y</code>	True if both <code>x</code> and <code>y</code> are true
<code>x or y</code>	True if either <code>x</code> or <code>y</code> is true
<code>not x</code>	True if <code>x</code> is false

```
# Example of the 'and' operator
age = 25
income = 2000

# Combining multiple conditions using the 'and' operator
if age > 18 and income >= 700:
```

```
print("This person is eligible to apply for a loan")
else:
    print("This person is not eligible to apply for a loan")
```

```
# Example of the OR operator
is_student = False
income = 100

if is_student or income < 250:
    print("You can get a discount")
else:
    print("You cannot get a discount")
```

```
# Example of the not operator
is_active = False

if not is_active: # Reverses the boolean value
    print("The account is inactive")
else:
    print("The account is active")
```

✓ 3.2. Loops (for Loop)

✓ 3.2.1 for Loops and range()

The `for` loop is used when you want to repeat the same process without writing it multiple times. You can specify the number of times you want to repeat using `range()`. Here is how you write a `for` loop using `range()`.

```
for <variable> in range(number of repetitions):
    <process>
```

Colons (:) and the initial indentation (4 spaces or a tab) are required, so be sure not to omit them. The processing will be repeated until the number of times specified in the `range` is completed. A variable is assigned values starting at 0, increasing by 1 with each iteration. If the variable does not have any special meaning assigned to it, common variable names like `i` (short for index or iterator), `j`, or `k` (alphabetical order after `i`) are often used for naming variables.

Next, we will show a code example that repeats an operation following the same structure, and then demonstrate how to simplify it using a `for` loop.

```
print('Python', 0)
print('Python', 1)
print('Python', 2)
print('Python', 3)
print('Python', 4)
```

```
Python 0
Python 1
Python 2
Python 3
Python 4
```

```
# Loop using for statement
for i in range(5):
    print('Python syntax', i)
```

```
Python syntax 0
Python syntax 1
Python syntax 2
Python syntax 3
Python syntax 4
```

```
# Using a for loop for iteration with specified start and end
s = 0
for n in range(1,11): # integers from 1 to (11-1)
```

```
s = s + n
print(s)
```

```
55
```

3.2.2. for Loops and Lists

The `for` loop can be combined with lists as discussed in Python Grammar II. You can iterate through the elements of a list using `range()`, but placing a list after `in` allows you to iterate through each element of the list sequentially. Here is how to write a `for` loop that iterates over a list.

```
for <variable> in <list>:
    <process>
```

The elements of the list are assigned to variables one by one, and the process is repeated for the number of elements.

Next, I will show examples combining `for` loops with lists and using `range()`.

```
# Loop and List
number = [1, 2, 3, 4, 5]
for n in number:
    print(n)
```

```
# Loop and List
number = [1, 2, 3, 4, 5]
s = 0
for n in number:
    s = s + n # Add elements of the list to s
print(s)
```

```
# Loop through a list using a for loop and range()
number = [1, 2, 3, 4, 5]
s = 0
for i in range(len(number)):
    s = s + number[i]
print(s)
```

3.2.3. for Loops and enumerate()

Sometimes, you may want to use both the index and the element when combining lists and for loops. In that case, `enumerate()` comes in handy. Here is how you can use `enumerate()` with a `for` loop on a list.

```
for <variable1>, <variable2> in enumerate(<list>):
    <process>
```

Note that `enumerate` is an English word that means to list items one by one.

Variable 1 stores the index of the list, variable 2 stores the elements of the list, and the process is repeated. Below are examples of code that extracts the index and elements of a list sequentially using `enumerate` and `range`.

```
menu = ['salad', 'soup', 'steak', 'cake']
for i, food in enumerate(menu):
    print(i, food) # Using enumerate() to extract index and element in order

0 salad
1 soup
2 steak
3 cake
```

```
menu = ['salad', 'soup', 'steak', 'cake']
for i in range(len(menu)):
    print(i, menu[i])
```

```
0 salad
1 soup
2 steak
3 cake
```

3.3. Practice Questions

(3-1) Write a program using the `if` statement.

Start coding or [generate](#) with AI.

(3-2) Write a program of your choice using a `for` loop.

Start coding or [generate](#) with AI.

(3-3) Let's assume that a variable `x` contains a list of integers with a length of 1 or more. Create a program that returns the largest element in `x`. Achieve this without using functions like `max()`, and instead use loops (repetition with `for` statement).

```
x = [1, 2, 3] # Input example
```

```
# WRITE ME!
```

4. Python Grammar IV

Related explanation video: https://www.youtube.com/playlist?list=PLIEF5y87wttyc3mVmmRpUcAi_a74w52Cl

4.1. Functions

There may be times when you want to reuse something you created earlier while creating a program. By using a **function**, you can name and save the flow of that process, allowing you to call it when needed. Such predefined processes to execute code are called functions.

The format is as follows:

- Function definition

```
def <function_name>(<arguments>, ...):
    <function_body>
```

- Function call

```
<function_name>(<arguments>, ...)
```

When calling a function, you pass 0 or more **arguments**. These argument values can be used in the processing within the function. To make this clearer, let us explain specifically using a program to find the absolute value as an example.

When given a value `x`, you can find its absolute value `y` using the program below.

```
# Program to find the absolute value
x = -1
if (x < 0):
    y = - x
else:
    y = x
print(y)
```

```
1
```

Let's turn this process into a function. To use a function, you first need to **define** the function. Function definition is done as follows:

```
# Definition of a function to find the absolute value
def absolute(x):
    if (x < 0):
        y = - x
    else:
        y = x
    return y
```

```
# Calling the function to find the absolute value
print(absolute(-1))
```

```
1
```

Here, the function name is defined as `absolute`, which means absolute value. When calling (using) this function, we use this name.

Let's call the `absolute` function. When you call a function, you must provide arguments, and their number has to match the number specified in the function's definition.

Earlier we mentioned that the code inside a function can use the values of its arguments. In the example above, when calling the function, we provide -1 as an argument, and this gets copied to the argument `x` that was specified in the function definition.

In programming terminology, the arguments provided during the function definition are called **formal parameters**, and the arguments provided during the function call are called **actual parameters**.

The value returned by the `return` statement within the function is known as the **return value**. In the example above, `y` becomes the return value, and you can confirm that it is the absolute value of `x`.

You can also define functions with multiple arguments. Just make sure that the order of the actual arguments corresponds to that of the formal parameters.

```
# WRITE ME!
```

```
# Answer: A function to calculate the power
def power(x, y):
    return x ** y
```

```
power(2, 3)
```

```
8
```

```
power(3, 2)
```

```
9
```

When explicitly specifying the correspondence between formal parameters and actual arguments, you specify it in the form `<formal parameter>=<actual argument>`. Arguments passed in this format are called **keyword arguments**. In contrast, arguments for which the correspondence is not explicitly specified (assuming that the order of the actual arguments corresponds to that of the formal parameters) are sometimes referred to as **positional arguments**.

```
power(y=2, x=3)
```

```
9
```

Furthermore, you can assign default values to some arguments, making those arguments optional. This is called **default arguments**. If an argument is omitted during a function call, the default value will be used.

```
def power(x=1, y=1):
    return x ** y
```

```
power(x=3)
```

```
3
```

4.2. Practice Problems

(4-1) Define a function `f` that takes a string `x` with a length of 1 or more as an argument and returns the last character of `x`.

```
x = 'python'
```

```
# WRITE ME!
```

(4-2) Define a function `f` that takes a dictionary as an argument `x` and returns the number of keys stored in `x`.

```
x = {"a": 1, "b": 2, "c": 3, 'd':4, 'e':5}
```

```
# WRITE ME!
```

(4-3) Define a function `f` that takes a list as an argument `x` and returns a list of elements of `x` in reverse order. However, achieve this without using functions like `reversed`, but using loops or list slicing.

```
x = [3, 9, 7, 1, 0]
```

```
# WRITE ME!
```

(4-4) The following function was intended to display "I am X years old". However, as it is, it results in an error. Please modify the function below. The argument for the function, `age`, is expected to be an integer. Assign the string you want to display to `msg`, store it in a dictionary. Then, print the value from the dictionary by specifying its key.

```
def print_my_age(age):
    msg = 'My age is ' + age + ' years old.'
    msg_dict = {'msg': msg}
    print(msg_dict['msg'])

print_my_age(18)
```

4.3 Modules and Libraries

When developing programs, there are times when you want to use functions or classes defined in other program files. In Python, you can do this by importing them as **modules**. A module represents a single program file and serves as a component.

A collection of modules is called a **package**, and a collection of packages is referred to as a **library**. The terminology around this is not clearly defined, and sometimes the terms "library" may also be used to refer to modules or packages. In the following discussion, we will use the terms "module" and "library". You can understand them to mean "a single program file" and "a collection of multiple program files", respectively.

In Python, you can read a module by importing it. If the program file is named `functions.py` in the same directory, you write it as follows.

```
import functions
```

When you `import` a program, you can use functions or classes defined in that program file. Let's try calling the function `f` defined in the `example.py` file. This `f` function is an identity function that takes one argument and returns it as is.

```
functions.Nibai(2)
```

As development progresses and the size of the program grows, it becomes important to divide it into smaller components for management. By grouping functions and classes by functionality and extracting them into separate files, the visibility of the program often improves. It is advisable to keep this in mind when undertaking large-scale development.

Next, let's talk about libraries. In Python, there is something called the standard library, which provides various functions that we can use. One of the modules included in the standard library is the `math` module, which contains mathematical functions. Here, we will discuss this module.

The `math` module defines exponential functions, logarithmic functions, trigonometric functions, and more. By importing this module, you can use these functions. When actually using them, you need to specify `<module_name>.<variable_name/function_name/class_name>`.

```
import math
```

```
# Exponential function
math.exp(1.0)
```

```
#Logarithmic Function
math.log(math.e)
```

By using `from <module_name> import <variable_name/function_name/class_name>`, you can use the variable/function/class without the module name.

```
from math import sin
from math import cos
from math import tan
from math import pi
```

```
# Trigonometric Function
sin(pi)
```

```
# Trigonometric Function
cos(pi)
```

```
# Trigonometric Function
tan(pi)
```

In addition to the standard library provided by Python, various libraries have been developed and publicly available. These libraries, which are distinct from the standard library, are referred to as external libraries. Python is one of the languages with the most comprehensive external libraries. In particular, it is popular for performing machine learning tasks in data analysis due to the abundance of libraries available. We will learn more about external libraries in the next section.